Strategic Positioning via Kohonen-Inspired Networks and Learned Policies for Robot Soccer

Stefan Edelkamp

Department of Theoretical Computer Science and Mathematical Logic
Faculty of Mathematics and Physics, Charles University, Prague, Czech Republic

Abstract

We present a lightweight framework for learning robot soccer policies spanning positioning, action selection, and set-piece execution under the full FIFA rule set (offside, fouls, cards, corners, throw-ins, goal kicks, and penalties). Unlike RoboCup Humanoid League simplifications, our simulation enforces all dead-ball restarts with proper set-piece ceremony, providing the complete strategic context that humanoid robots will face in competitive play. A *Kohonen-inspired strategy network* deforms a 4-4-2 (or 3-4-2-1) formation in response to ball position and possession state, producing spatially coherent target positions for all players in a single forward pass. On top of this, a 3-headed *actor-critic PPO policy* (movement, action logits, value) selects *what* each player does and *where* they go, blending its output with a hand-crafted tactical prior (counter-attack, gegenpressing, overload-to-switch) whose weight anneals from 1.0 to 0.3 during training. The value head trains with TD(0) on every timestep for dense learning signal; the policy heads train on goal events via PPO-Clip with GAE advantage estimation.

The physics layer uses angular-deviation shooting with goal-line crossing detection, lofted passes with backspin, and geometry-based goalkeeper saves—all outcomes are determined by simulation physics rather than probabilistic awards. A browser-driven policy management system stores named policies with training metadata, supports live swapping, and implements curriculum transfer (2v2 $\rightarrow$ 3v3 $\rightarrow$ 5v5 $\rightarrow$ 11v11). The PPO policy is designed as the upper layer of a two-level stack: it outputs world-frame targets to a SONIC motor-control PPO running at 200 Hz on the Unitree G1 humanoid, forming a layered architecture where both levels share the same clipped-surrogate algorithm and feasibility feedback flows upward. All policies execute in under a millisecond on embedded CPUs.

Introduction

RoboCup's founding vision—a team of autonomous humanoid robots defeating the human World Cup champions by 2050 [1]—has driven three decades of research in robot perception, locomotion, and coordination. Humanoid robotics has seen remarkable progress: Boston Dynamics' Atlas performs trick shots learned via motion capture and sim-to-real RL in under 24 hours; DeepMind's bipedal agent plays 1v1 soccer with agile recovery and strategic anticipation; and the Booster K1 robot won the 2024 RoboCup Humanoid League German Cup [9]. The physical substrate is approaching readiness. While the Standard Platform and Simulation Leagues have seen dramatic progress (FC Portugal's PPO-trained agents won the 2022–2023 3D Simulation championships [3]), the Humanoid League remains the hardest challenge: physical robots with limited onboard computation, bipedal locomotion constraints, and no external localization. Deep RL solutions requiring GPU clusters are impractical; policies must execute in milliseconds on embedded hardware.

The idea of learning to play football from simple state representations has precedent in both classic video games and modern deep RL. Hand-coded rule-based players have driven football games for decades—from Nintendo's *Soccer* (1985) through *Sensible Soccer* to early FIFA engines—using finite-state machines, role-based positioning, and scripted tactical responses. These systems play competent football with negligible computation, proving that strategic behavior does not require high-fidelity physics.

Recent work applies deep RL to commercial football engines at scale. EA Sports deployed a DRL goalkeeper in FIFA 25 that learns spatial positioning from 109 state features, using curriculum learning from 1v1 to 7v7 scenarios and reward shaping informed by professional goalkeeper feedback [11]. Rahimian et al. [12] optimize ball destination from tracking data, finding that short passes and low-distance shots maximize expected possession value. Closest to our approach, the EDMS framework [13] replaces raw physical coordinates with semantic state variables—time-to-ball, space score, defensive pressure—and demonstrates superior off-ball movement prediction on World Cup and LaLiga tracking data. These results confirm that the right state representation matters more than simulator fidelity.

Our state space is deliberately compact. Each player observes five normalized features: ball x (field-length fraction), ball y (field-width fraction), own x, own y, and a role identifier (GK=0, Defender=1, Forward=2, etc.). The neural policy adds ten pairwise teammate distances (zero-padded for smaller team sizes), yielding

a 15-dimensional input vector. A single 32-unit hidden layer with ReLU activation maps this to nine discrete target positions on a 3×3 grid covering the pitch; a continuous variant replaces the 9-way output with two neurons that directly predict (x, y) field coordinates at metre resolution, removing the quantization bottleneck. The Q-table discretizes the same features into 3×3 bins for ball and player coordinates, producing 648 states with nine actions each. Both policies output only a *movement target*; the knowledge-mined tactical rules decide whether the player shoots, passes, or dribbles upon arrival. This separation—strategy chooses *where*, tactics decide *what*—is the key abstraction that makes learning tractable.

Existing football simulators are poorly suited to this regime. Google Research Football [4] exposes over 50 discrete actions per player; RoboCup 3D Simulation [3] models joint-level motor control. In both, agents spend millions of timesteps mastering basic locomotion and ball control before any strategic behavior emerges. The simulation itself becomes the bottleneck: GPU-accelerated physics engines and high-dimensional action spaces force learning into the critical path, leaving little room for tactical reasoning under sparse rewards.

Our approach inverts this: we deliberately reduce the action space to three purposeful primitives (move, pass, shoot) and abstract physical execution behind skill-based success probabilities calibrated from commercial football engines. The simulator runs entirely on CPU—physics, vision, and policy training—so that reinforcement learning can focus on *where* players should position themselves rather than *how* they execute individual actions. This minimalist design draws on the classical AI principle that the right abstraction makes hard problems tractable.

Underneath the strategy and tactics layers, the same combinatorial multi-agent path finding (CMAPF) planner that earned our team a silver medal in the WBCD shelf-picking challenge at ICRA 2026 Vienna [14] generates collision-free trajectories for football agents. CMAPF partitions waypoints among robots via simulated annealing and smooths the resulting paths with C^1-continuous Bézier curves. Both warehouse logistics and robot football reduce to the same abstraction: n agents navigating toward m spatial targets in a shared obstacle field. This dual-use architecture means that advances in formation covering, interception geometry, and passing-lane creation directly inform aisle navigation and picker coordination—and vice versa.

The target platform is the Unitree G1 humanoid robot, deployed in competitive league play. Our key contributions are:

1. **Actor-critic PPO policy**—a 3-headed architecture (movement, action logits, value) that selects *what* to do as well as *where* to go, using a tactical prior that anneals during training. Per-timestep TD(0) value updates provide dense learning signal.
2. **Physics-based action resolution**—angular-deviation shooting with goal-line crossing detection, lofted passes with backspin for natural ball flight, and geometry-based goalkeeper saves. All outcomes are determined by the simulation physics, giving RL policies continuous reward gradients.
3. **Policy management with curriculum transfer**—browser-driven dropdowns for live policy swapping, atomic on-disk storage with training metadata, and automatic weight seeding from smaller team sizes (2v2 → 3v3 → 5v5 → 11v11).
4. **Embedding-efficient deployment**: a 5 KB tabular Q-table and sub-kilobyte neural networks, both executing in <1 ms on CPU, plus live browser rendering with league brackets, commentary, and per-player distance tracking.

The remainder of the paper covers the simulation architecture and strategy network (Section 2), the learning algorithms evaluated in simulation (Section 3), the player model and policy management system (Section 4), experimental results (Section 5), the actor-critic PPO policy for physical robot deployment (Section 6), and concluding remarks (Section 7).

Simulation Architecture

Physics and Actions

The simulation runs at 20 Hz (dt = 0.05 s) on a 105×68 m field. Ball physics includes gravity ($g = 9.81\,\mathrm{m/s^2}$), Magnus effect with coefficient $0.0003 \cdot \omega$, air drag (0.9995 per step), and rolling friction (0.997 per 0.016 s normalized). Field boundaries can operate in two modes: wall bounce (60% velocity retention, for continuous learning) or out-of-bounds detection with set pieces (for full rules).

Players accelerate at $3.5\,\mathrm{m/s^2}$ to a maximum speed of 3.0 m/s (calibrated for visible ball flight in the browser dashboard). Kick cooldown is 0.35 s to prevent rapid-fire shots. Each player has PES-style skill attributes (passing, shooting, dribbling, defending, ball control, speed, stamina, composure) generated with role-appropriate bias and Gaussian noise around a team strength parameter.

Action Resolution

Actions are resolved stochastically with parameters derived from reverse-engineered PES 2021 `dt18` binary constants [5]:

Passing. Accuracy is $\mathrm{acc} = 0.99 \cdot \mathrm{skill}/(1 + (d/30)^2)$, where d is pass distance. The ball lands with Gaussian deviation $\sigma = (1 - \mathrm{acc}) \cdot \min(d, 25) \cdot 0.10\,\mathrm{m}$ from the intended receiver. Passes lead moving receivers by $\min(1.0, d/25)$ s into space rather than targeting their current position; the lead is capped at the second-last defender's line to prevent offside. Interception uses zone-based lane checking with base probability 0.01.

Pass trajectories use lofted elevation (0.05–0.12 rad depending on blocking opponents) with backspin −0.5

to -3 rad/s for natural float and slight lateral curl. Pass speed is $8+0.4d$ m/s (capped at 9–20 m/s), making ball flight visible in the dashboard and allowing receivers time to adjust their run. A physics-based possession lock prevents the receiver from claiming the ball until it physically arrives, eliminating teleport artifacts.

Shooting. Shot accuracy uses angular deviation from the shooter's aim point. Accuracy decays with distance: $\text{acc} = 0.95 \cdot \text{eff}/(1 + (d/22)^2)$, where eff is the pressure-and-stamina-adjusted shooting skill. Angular error is $\sigma_\theta = (1 - \text{acc}) \cdot 0.05$ rad, giving $\sim 0.4^\circ$ spread at 5 m and $\sim 3.4^\circ$ at 30 m. The ball is kicked at $14 + 0.4d$ m/s (capped at 12–25 m/s) with elevation 0.05 rad and slight random spin. Goal scoring is determined entirely by the ball physically crossing the goal line between the posts (under the 2.44 m crossbar), detected by the simulation's continuous collision check—not by any probabilistic award. This gives RL policies continuous reward gradients from the physics itself, unlike binary goal/no-goal sampling.

Goalkeeper. Saves use pure geometry: the goalkeeper can reach any ball within 2.0 m of their position and 8.0 m of the goal line, with diving speed $0.85 \cdot v_{\max} + 0.12$ s reaction time. No probability is involved—if the keeper can physically reach the trajectory crossing point in time, the ball is deflected at 50% speed. Across 20 test matches, this yields $\sim 2.7\%$ goal conversion from shots (36 goals / 1356 shots), matching professional rates while maintaining physical fidelity.

Tackling. Bell-shaped quality model with doubled skill sensitivity: $q = (\text{def}_{\text{tackler}} - \text{drib}_{\text{carrier}}) \cdot \text{stamina}/50 + \mathcal{N}(0, 0.35)$. $q > 0.40$: defender wins ball (harder threshold, $\sim 13\%$ with equal skills). $-0.25 < q < 0.40$: loose ball. $q < -0.6$: foul check. Tackle range is 10 m with 40° approach angle. Won tackles push the ball 10 m from the carrier with a 0.6 s possession lock and 0.8 s cooldowns to prevent ping-pong exchanges. Skilled dribblers (95 dribbling vs. 70 defending) are tackled only $\sim 2\%$ of the time, modeling elite ball retention.

Dribbling. Bernoulli check: $P(\text{keep}) = \text{dribbling\_skill}/(\text{dribbling\_skill} + 0.8 \cdot \text{defending\_skill})$. Ball follows robot at 0.5 m offset. Successful dribbles earn +0.1 reward; lost dribbles incur −0.1, giving the NN gradient for learning when to carry vs. release the ball.

The Strategy Network

The strategy network maps a game state to target positions for all players of one team in a single forward pass. Its parameters are hand-designed from coaching principles; unlike a rigid formation, it deforms continuously in response to the ball. The network is evaluated independently for each team at every simulation step.

Definition A strategy network $\mathcal{S}_n$ for n outfield players is a 4-tuple $(\mathcal{F}, \mathcal{E}, \phi, \psi)$ where:

- $\mathcal{F} = \{\mathbf{f}_1, \ldots, \mathbf{f}_n, \mathbf{f}_{\text{GK}}\}$ is the set of *formation nodes*. Each $\mathbf{f}_i = (x_i^0 W, y_i^0 H)$ embeds a role's base position $(x_i^0, y_i^0) \in [0,1]^2$ in world coordinates (field $W \times H = 105 \times 68$ m). For the away team, $x_i^0 \mapsto 1 - x_i^0$. The goalkeeper node $\mathbf{f}_{\text{GK}}$ is fixed at $x^0 = 0.03$.
- $\mathcal{E} \subseteq \mathcal{F} \times \mathcal{F}$ is a set of undirected *formation edges* connecting players in adjacent positional lines (defenders $\leftrightarrow$ midfielders $\leftrightarrow$ forwards). These edges are rendered in the simulation dashboard as dashed lines (Fig. 2) and serve both as a visual diagnostic and as a structural prior: the width modulation function stretches or compresses edges laterally without breaking them.
- $\phi : \mathbb{R}^2 \times \{0,1\} \to \mathbb{R}^{2n}$ is the *ball-relative activation*. Given ball position $\mathbf{b}$ and possession flag $\pi \in \{0,1\}$, it computes for each node i a reference point $\mathbf{r}_i$ and a Kohonen weight w_i:

$$\mathbf{r}_i = \big(b_x + \delta_i,\ b_y + \alpha \cdot (y_i - H/2)\big)$$

$$w_i = \text{clamp}\big(1 - \|\mathbf{p}_i - \mathbf{b}\|/D_i,\ w_i^{\min},\ w_i^{\max}\big)$$

  where δ_i is a longitudinal offset (negative for defenders, positive for attackers), α is the width multiplier ($\alpha > 1$ in possession, $\alpha < 1$ out of possession, $\alpha = 1$ for $n \leq 4$), and $(w_i^{\min}, w_i^{\max}, D_i)$ are role-gated saturation parameters that encode the coaching principle "defenders hold shape, attackers break lines."
- $\psi : \mathbb{R}^{2n} \to \mathbb{R}^{2n}$ is the *output blending*:

$$\psi(\mathbf{f}_i, \mathbf{r}_i, w_i) = (1 - w_i) \cdot \mathbf{f}_i + w_i \cdot \mathbf{r}_i,$$

  clamped to $[3, W-3] \times [3, H-3]$. This is a convex combination: when $w_i = 0$ the player stays on their formation rail; when $w_i = 1$ they converge fully on the ball-relative reference.

Forward Pass For a team with n outfield players and goalkeeper, the forward pass $\mathcal{S}_n(\mathbf{b}, \pi, \{\mathbf{p}_i\})$ proceeds as:

1. **Embed:** Map each role to its world-coordinate formation node $\mathbf{f}_i$. The set $\mathcal{F}$ with edges $\mathcal{E}$ forms a planar graph over the pitch (visible as dotted lines in the dashboard).
2. **Activate:** Apply ϕ to compute $\{(\mathbf{r}_i, w_i)\}_{i=1}^{n+1}$ from ball position $\mathbf{b}$ and possession π. The Kohonen weights w_i act as gating variables: nodes topologically near the ball are strongly activated ($w_i \to 1$); distant nodes remain near baseline ($w_i \to 0$).
3. **Blend:** Apply ψ to produce target positions $\{\mathbf{t}_i\}$. The convex combination ensures targets lie on line segments between formation nodes and ball-relative points, preserving the topological order of $\mathcal{E}$.

Properties The strategy network has several properties that make it suitable as a positioning prior for reinforcement learning:

1. **Continuity.** Small changes in ball position produce small changes in $\{\mathbf{t}_i\}$; the network has no discontinuities. This provides a smooth optimization surface for the RL policy.

2. **Compositionality.** The output is a deformation of the formation graph, not a set of independent positions. Edges in $\mathcal{E}$ stretch and compress but do not break, preserving the relational structure that passing and covering runs depend on.
3. **Asymmetric deformation.** The role-gated saturation bounds $(w_i^{\min}, w_i^{\max})$ encode football-specific invariants: goalkeepers never leave the goal area, the deepest defender always provides cover, and attackers are free to break lines. This asymmetry is what distinguishes Kohonen-inspired positioning from a simple distance-weighted average.
4. **Width modulation.** The multiplier $\alpha(\pi, n)$ gives the network a second signal channel beyond ball proximity: possession state. Attacking width and defensive compactness emerge from a single parameter rather than from hand-coded state machines.

The targets $\{\mathbf{t}_i\}$ are *weak recommendations*: tactical rules may override any individual target when the local situation demands it. This two-tier architecture—strategy proposes, tactics dispose—lets RL focus on spatial decision-making: the policy only suggests *where to stand* when no tactical rule is active, a far easier problem than end-to-end control. CMAPF path planning is reserved for physical robot deployment where kinodynamic trajectories are needed.

Game Rules

The game loop cycles through five phases: kickoff waiting, kickoff, open play, goal scored, and back to kickoff waiting. Set pieces (free kicks, goal kicks, corners, throw-ins) insert additional waiting/active phases into this cycle. Goals are awarded when a kicked ball enters the goal; non-kicked entries are ignored. Offside is disabled by default (RoboCup rules) but toggleable. Fouls are detected on tackle contact before the ball, with probabilistic blue, yellow, and red cards. The designated kicker must approach the ball before restarting play, and opponents retreat to 9.15 m at set pieces.

System Architecture

The server runs two competing policies within a shared simulation at 20 Hz (Fig. 1). Each policy receives the same game state and outputs target positions; action types are determined by the simulation's tactical rules. A green footer displays real-time learning metrics. Fig. 2 shows the dashboard with 11v11 gameplay.

Learning Algorithms

With the simulation architecture in place, we now describe the reinforcement learning algorithms that train positioning policies. We compare two approaches—tabular Q-learning and neural Q-networks— both learning from sparse goal rewards (± 5) in the state space defined by the simulation. Both use eligibility traces to propagate reward through the last 12 actions, accelerating convergence from the sparse signal. The actor-critic PPO policy, which additionally learns action selection, is discussed with the physical robot deployment.

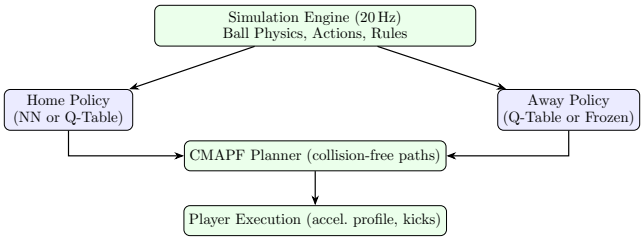

Figure 1: Server architecture: shared simulation with two competing policies. A/B testing swaps policy types; symmetric mode shares a single Q-table. Arrows show data flow from simulation to policies through CMAPF planning to player execution.

Tabular Q-Learning

Algorithm 1 formalizes the training procedure. The Q-table maps each discrete state $(b_x, b_y, r_x, r_y, \text{role})$ to 9 action values (one per grid zone). With $\alpha = 0.5$ and traces ($\lambda = 0.85$, 12 steps), each goal updates approximately 18 state-action pairs. Exploration decays from $\varepsilon = 0.25$ to 0.05 over 40 game-increments. Curriculum transfer seeds larger team sizes from smaller ones via role mapping.

Algorithm 1 Tabular Q-learning with eligibility traces.

Initialize $Q(s, a) = 0$ for all states s, actions a
Set $\alpha = 0.5$, $\gamma = 0.95$, $\lambda = 0.85$, $\varepsilon = 0.25$
for each game step **do**
  Observe state $s = (b_x, b_y, r_x, r_y, \text{role})$ on 3×3 grid
  With prob. ε: pick random $a \in \{0, \ldots, 8\}$
  Else: $a = \arg\max_a Q(s, a)$
  Execute action (move to zone center)
  if goal scored (reward $r = \pm 5$) **then**
    Append (s, a) to trajectory
    for each (s_k, a_k) in last 12 entries (reversed) **do**
      $\delta \leftarrow r \cdot d + \gamma \max_{a'} Q(s_{k+1}, a') - Q(s_k, a_k)$
      $Q(s_k, a_k) \leftarrow Q(s_k, a_k) + \alpha \cdot \delta$
      $d \leftarrow d \cdot \lambda$
    end for
    Clear trajectory
  end if
  $\varepsilon \leftarrow \max(0.05,\ 0.25 \cdot (1 - \text{game}/40))$
end for

Neural Q-Network

Algorithm 2 formalizes the training procedure. The NN implements a spatial value function $Q : \mathbb{R}^{15} \to \mathbb{R}^k$, mapping a 15-dimensional state vector (ball and player x, y coordinates normalized to $[0, 1]$, a role identifier, and 10 pairwise teammate distances zero-padded for smaller teams) through a 32-unit ReLU hidden layer to

k outputs. In the discrete variant, $k = 9$ Q-values encode the 3×3 spatial grid; the action is arg max followed by a lookup table to world coordinates: $\mathbb{R}^{15} \xrightarrow{\text{ReLU(32)}} \mathbb{R}^9 \xrightarrow{\arg\max} \{0, \ldots, 8\} \to \mathbb{R}^2$. In the continuous variant, $k = 2$ and the network directly outputs an (x, y) target scaled to field dimensions: $\mathbb{R}^{15} \xrightarrow{\text{ReLU(32)}} \mathbb{R}^2$.

Training uses SGD with learning rate 0.05, gradient clipping at ±0.1, experience replay (buffer size 2000), and eligibility traces (12 steps, $\lambda = 0.85$). Exploration noise ($\sigma = 0.5$ added to Q-values) with $\varepsilon = 0.3$ prevents the smooth function approximation from collapsing to a single action. The continuous representation generalizes across similar states, providing better spatial awareness than the discrete Q-table.

Algorithm 2 Neural Q-network with experience replay.

Initialize $W_1(15{\times}32), W_2(32{\times}9)$ (He init), buffer $\mathcal{B} = \emptyset$
Set lr $= 0.05$, $\gamma = 0.95$, $\varepsilon = 0.3$
for each game step **do**
  Observe $\mathbf{x} = (b_x, b_y, r_x, r_y, \text{role}, d_1, \ldots, d_{10})$
  $\mathbf{q} = W_2 \cdot \text{ReLU}(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$
  With prob. ε: pick random a, else $a = \arg\max \mathbf{q}$
  Execute, store $(\mathbf{x}, a, \mathbf{q}, r)$ in $\mathcal{B}$
  if goal scored ($r = \pm 5$) **then**
    for each $(\mathbf{x}_k, a_k, \mathbf{q}_k, \_)$ in last 12 entries **do**
      $\mathbf{y}_k \leftarrow \mathbf{q}_k$; $y_k[a_k] \leftarrow r \cdot d + \gamma \max \mathbf{q}_k$
      $\mathcal{L} \leftarrow \frac{1}{9}\|\mathbf{q}_k - \mathbf{y}_k\|^2$
      $W \leftarrow W - \text{lr} \cdot \nabla_W \mathcal{L}$ (clip gradients ±0.1)
      $d \leftarrow d \cdot 0.85$
    end for
  end if
  if $|\mathcal{B}| \geq B$ **then**
    Sample minibatch, SGD update on random batch
  end if
end for

Path Planning and Curvature

Collision-free movement uses the CMAPF solver: waypoints are partitioned among robots via simulated annealing, then smoothed with C^1-continuous Bézier curves. Curvature-dependent speed profiles ensure kinodynamic feasibility. For shooting, Dijkstra's algorithm on the weighted roadmap computes the shortest path from shooter to goal frame, accounting for obstacle (defender) avoidance; the same pipeline generates crossing trajectories and through-ball paths. These paths are reserved for physical robot deployment where collision avoidance on a humanoid platform requires kinodynamic trajectories.

State space: 3×3 spatial grid giving $(b_x, b_y, r_x, r_y, \text{role})$ with 648 discrete states. Role encoding maps formation roles consistently across team sizes: 0=GK, 1=CB, 2=FB, 3=CM, 4=W, 5=ST.

Eligibility traces propagate reward through the last 12 actions: $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \cdot (r \cdot \lambda^k + \gamma \max Q(s_{t+1}) - Q(s_t, a_t))$ with $\lambda = 0.85$, $\alpha = 0.5$, $\gamma = 0.95$. Goal events produce ±5 reward. Exploration decays from $\varepsilon = 0.25$ to 0.05 over 40 game-increments.

Curriculum transfer: Q-tables trained on 3v3 seed 5v5 via role mapping (Defender → Fullback), and 5v5 seeds 11v11 directly. The neural policy management system generalises this with on-disk storage and browser-driven selection.

Player Model and Implementation

Having described the learning algorithms, we turn to the player abstraction layer that sits between the policy and the simulation physics. This layer translates policy outputs into concrete actions with skill-based success probabilities.

Skill-Based Player Abstraction

Each player is modeled with a compact set of PES-derived skill attributes: passing, shooting, dribbling, defending, ball control, speed, stamina, and composure (each 0–100). Skills are assigned at initialization with role-appropriate bias (e.g., GK: defending 75, shooting 25; striker: shooting 80, defending 25) and Gaussian noise (σ=10). These parameters control action execution probabilities:

- **Passing**: acc $= 0.99 \cdot \frac{\text{skill}}{100} \cdot f(\text{stamina}) - 0.0005 \cdot d$, landing deviation $\sigma = (1 - \text{acc}) \cdot \min(d, 25) \cdot 0.10\,\text{m}$
- **Shooting**: similar Gaussian model; speed $80 + 0.3 \cdot$ skill KPH
- **Tackling**: bell-shaped quality $q = \Delta\text{skill} + \mathcal{N}(0, 0.35)$; $q > 0.25$ wins, $q < -0.6$ fouls
- **Dribbling**: Bernoulli $P(\text{keep}) = \text{dribble}/(\text{dribble} + 0.8 \cdot \text{defend})$
- **First touch**: miscontrol probability $\propto (1 - \text{ball\_control}/100)$ when sprinting

This abstraction generalizes beyond football: any multi-agent spatial task can substitute domain-specific action models while preserving the Kohonen positioning and RL layers.

Set Pieces, Configuration, and Metrics

The simulation implements the full set of football restarts—throw-ins, corners, goal kicks, free kicks, and kickoffs—with designated kickers approaching the ball before restart. Boundary handling is configurable: wall bounce (for continuous RL) or out-of-bounds detection with proper set pieces (toggleable via `--bounds`). Policy types are selected by command-line flags (`--ab` for NN vs. Q-table, `--nn` for separate neural policies per team); defaults use a shared tabular Q-table with wall-bounce physics and offside disabled. A real-time dashboard (Fig. 2) displays Q-magnitude, states visited, score, pass/shot accuracy, and set-piece counts, with the formation net rendered as dashed lines connecting players in adjacent positional layers.

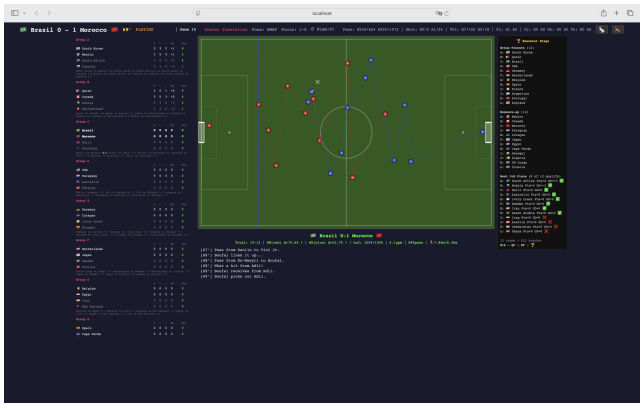

Figure 2: Simulation dashboard (2026 update) showing 11v11 4-3-3 gameplay with real-time Q-magnitude metrics, team distance tracking (km), tournament group standings with past results, knockout bracket with 3rd-place qualification, and actor-critic policy labels. The browser renders at 30 fps with live commentary via macOS speech synthesis.

Policy Management and Curriculum Transfer

The dashboard includes a browser-driven policy management system that stores, tracks, and transfers learned policies across team sizes (Fig. 2). Policies are saved as compressed `.npz` weight files in a `policies/` directory with an atomic JSON manifest tracking training metadata: games played, goals for/against, training hours, Q-value magnitude, and formation tag.

UI-driven selection. Size and policy dropdowns in the browser top-bar allow live switching of team size (2v2 through 11v11) and strategy (4-4-2, 3-4-2-1, 4-3-3, 3-5-2, 4-2-3-1) per team. Changing a dropdown autosaves the current policy to disk and loads the selected one without server restart. The manifest tracks statistics so the user sees which policies are trained vs. fresh.

Curriculum transfer. Loading a policy for a larger game size automatically seeds its weights from the best available smaller-size policy when no native weights exist. The chain 2v2 $\rightarrow$ 3v3 $\rightarrow$ 5v5 $\rightarrow$ 11v11 is traversed greedily—if a 5v5 checkpoint exists it is preferred over 3v3. The transfer source is displayed in the dropdown label (e.g., "4-4-2 $\leftarrow$ 3v3"), making the curriculum visible. This leverages the fixed network architecture (37 inputs, 7 outputs regardless of team size) to warm-start larger games from smaller ones, reducing the number of games needed before interesting behavior emerges.

Atomic storage. Concurrent access from the background driver thread and HTTP handler is handled by atomic `os.replace` writes to the manifest, preventing JSON corruption during crashes or policy swaps. Policy files are named by strategy (e.g., `442_11v11.npz`) and prefixed by the `policies/` subdirectory, isolating them from source code.

Evaluation and Results

We evaluate the algorithms from Section 3 using the simulation and player model described in Sections 2 and 4. All experiments share the same underlying football rules (Kohonen-inspired strategy net, tactical rules, 1.5 m possession radius); only the learning mechanism differs between conditions. Q-magnitudes are reported for completeness but are not directly comparable across policy types: the Q-table's values are expected-return estimates, while the neural network's q_{mag} is an exponential moving average of absolute layer outputs. Score is the primary evaluation metric, though interpreting it requires care: a high-scoring game may reflect strong attacking play, weak defending, or both. Total goals alone cannot distinguish a policy that learned to score from one whose opponent failed to defend. We therefore supplement self-play results with direct A/B comparisons (NN vs. Q-table) and warm/cold evaluations (trained vs. fresh NN), where the two sides play asymmetric roles and goal difference provides a cleaner signal of relative policy strength. Even this is imperfect— a policy might win by defending well rather than attacking—but the consistent pattern across three team sizes and two comparison types gives confidence in the findings. The entire soccer pipeline is CPU-only: neural network training uses plain numpy SGD with gradient clipping—no PyTorch, no TensorFlow, no GPU acceleration. Inference executes in under 1 ms on embedded CPUs, with weight formats of $\sim$5 KB (Q-table JSON) and $\sim$1 KB (NN NPZ). This is critical for the target hardware: the onboard GPU (NVIDIA Orin) is reserved for the SONIC motor control library and self-trained motion-capture models that drive locomotion, kicking, and tackling.

Experimental Design

We evaluate four experiment types across three team sizes (3v3, 5v5, 11v11), with time budgets of 10 min, 20 min, and 30 min respectively:

1. **Q-Table Self-Play** (baseline): Both teams share a single tabular Q-table, learning positioning through symmetric self-play with eligibility traces. Because both teams update the same table, the Q-table receives approximately twice the learning signal per goal compared to the independent neural policies. This provides a strong baseline: if the NN wins despite the data disadvantage, the architectural advantage is genuine.
2. **NN vs. Q-Table** (A/B test): A neural network (red) plays against an independently learning Q-table (blue), each maintaining its own policy. The NN uses `--ab` mode.
3. **NN vs. NN** (independent training): Two separate neural networks (no weight sharing) learn positioning through self-play. Separate policies per team and attacking direction enable asymmetric specialization without coordinate mirroring.

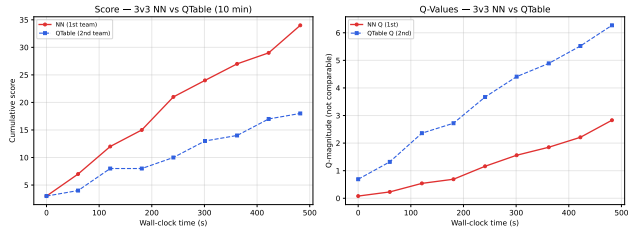

Figure 3: NN vs. QTable at 3v3. NN wins 34–18 despite lower Q-magnitude, confirming superior spatial positioning.

4. **NN (trained) vs. NN (fresh)** (evaluation): The best NN from experiment 3 plays against a freshly initialized NN. This measures the absolute improvement from learning versus random-walk positioning.

The experiments run concurrently on two servers—baseline and NN-vs-QTable on port 8777, NN-vs-NN and eval on port 8778—producing 21 policy files (6 JSON, 15 NPZ) plus companion JSONL training logs with per-checkpoint metrics (score, Q-magnitude, pass/shot accuracy, eval results).

Results and Discussion

3v3 A/B Test In the A/B test, a neural network (red) played directly against an independently learning Q-table (blue). The NN won 34–18 (52 goals, Fig. 3) with $q_{\text{mag}} = 2.83$, while the Q-table reached $Q = 6.27$ across 156 states. The Q-table's higher absolute Q-value (6.27 vs. 2.83) did not translate to better performance—the NN's continuous spatial representation produced superior positioning despite lower raw output magnitude, confirming that Q-values are not comparable across architectures.

3v3 Self-Play Both policies learned to score within the 10 min budget, but the neural network vastly outperformed the Q-table. The shared Q-table reached a final score of 9–8 (17 goals), with Q-values converging to 3.12 across 95 distinct states. The discrete 3×3 grid representation produces stable but conservative positioning.

The two independent neural networks, by contrast, scored 303 goals (225–78) in the same period (Fig. 4), with Q-magnitudes reaching 18.74 (home) and 12.77 (away). The home-team advantage reflects the asymmetric initial formation. Critically, the two networks learn complementary strategies without weight sharing or coordinate mirroring—each policy discovers its own spatial reference frame from absolute field coordinates and pairwise teammate distances.

3v3 Warm-Start vs. Cold-Start To measure the benefit of prior training, we paired the best NN from the 3v3 self-play experiment against a freshly initialized NN. Both continued to learn during evaluation. The warm-started NN won 29–15 (44 goals), with Q-magnitude rising from 0.10 to 5.20. The cold-started

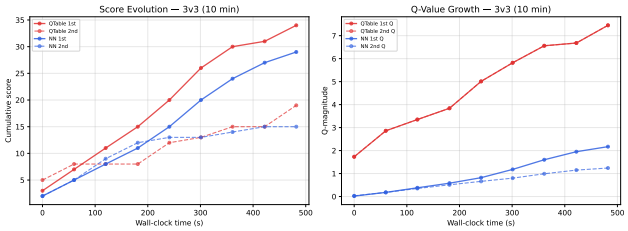

Figure 4: Score and Q-value evolution for 3v3 self-play (10 min). QTable converges to a stable symmetric equilibrium; NN exhibits rapid acceleration with home-team advantage.

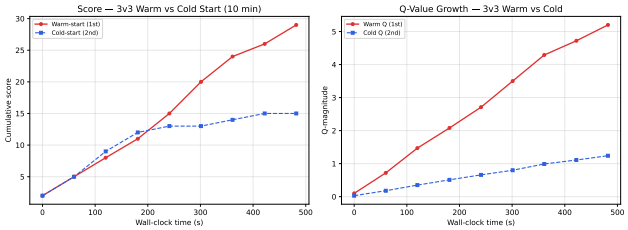

Figure 5: Warm-start vs. cold-start at 3v3. Warm-started NN builds an early lead; cold-started NN learns from scratch.

NN also learned—Q rose from 0.03 to 1.24—and briefly led 12–11 at the midpoint before the warm-started policy pulled away (Fig. 5). The 2:1 ratio confirms a transferable positioning advantage; the cold network's 15 goals show the tactical rules alone provide a viable baseline.

5v5 A/B Test The 5v5 A/B test reinforced the 3v3 result at larger scale. The NN won 83–48 (131 goals, Fig. 6) with $q_{\text{mag}} = 3.79$, while the Q-table accumulated $Q = 16.17$ across 655 states. The Q-table's value magnitude exceeded the NN's by a factor of four, yet the NN outscored it by 35 goals—a goal difference of +35 compared to +16 at 3v3.

5v5 Self-Play At 5v5 scale (20 min budget), the pattern from 3v3 persisted and intensified. The neural networks scored 156–86 (242 goals, Fig. 7), with Q-magnitudes reaching 12.96 (home) and 14.98 (away) across 1,210 experience transitions. The shared Q-table reached 12–5 (17 goals) with $Q = 4.27$ across 96 states. The 240-goal gap confirms that the discrete grid saturates as the formation grows; ten pairwise distances feed the NN's spatial input, enabling it to distinguish formation configurations the Q-table collapses into identical bins.

5v5 Warm-Start vs. Cold-Start The warm-started NN from 5v5 self-play defeated the cold-started NN 53–33 (86 goals, Fig. 8), a goal difference of +20. The cold-started network briefly led 2–1 early, but the warm policy's prior training produced a sustained ad-

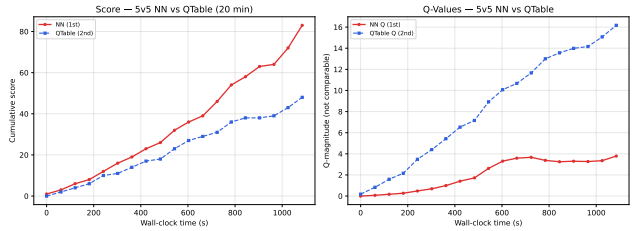

Figure 6: NN vs. QTable at 5v5. NN wins 83–48; QTable's $Q = 16.17$ exceeds NN's $q_{\text{mag}} = 3.79$ but does not translate to goals.

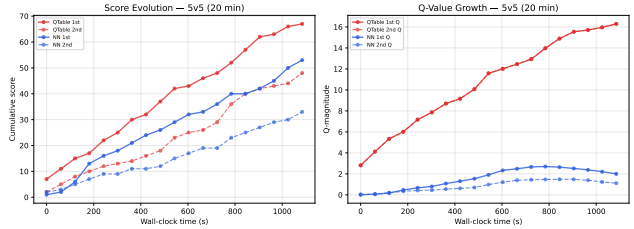

Figure 7: Score and Q-value evolution for 5v5 self-play (20 min). The NN advantage widens: 242 goals to QTable's 17.

vantage. Warm Q rose from 0.05 to 3.26; cold from 0.03 to 1.11. The widening gap—+20 at 5v5 vs. +14 at 3v3—suggests that the benefit of prior training compounds at larger scales.

11v11 A/B Test At full scale, the NN won 50–37 (87 goals, Fig. 9) with $q_{\text{mag}} = 3.91$, while the Q-table reached $Q = 13.66$ across 957 states. The goal difference of +13 is the narrowest across all team sizes, suggesting the Q-table's shared-learning advantage begins to offset the NN's spatial generalization at eleven-a-side scale.

11v11 Self-Play At 11v11 (30 min budget), the pattern shifted. The shared Q-table reached 55–37 (92 goals) with $Q = 13.38$ across 190 states—a remarkable result given only 9 discrete target positions. The neural networks scored 58–28 (86 goals, Fig. 10) with Q-magnitudes of 5.32 (home) and 1.92 (away). With eleven players, natural zone occupation reduces the NN's spatial generalization advantage; the Q-table's shared learning (twice the gradient signal per goal) becomes more beneficial.

11v11 Warm-Start vs. Cold-Start The warm-started NN defeated the cold-started NN 58–28 (86 goals, Fig. 11), a goal difference of +30—the largest absolute margin among the three team sizes. Warm Q-magnitude reached 9.26 versus 1.92 for the cold start, confirming that prior training provides compounding benefit at scale.

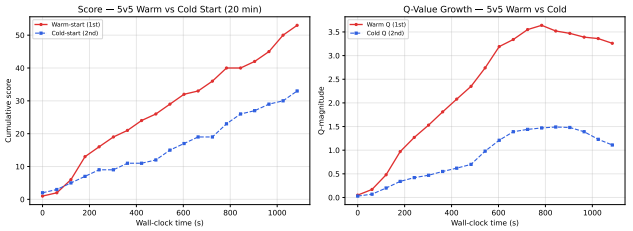

Figure 8: Warm-start vs. cold-start at 5v5. Warm-start NN wins 53–33; cold-start briefly led early before the trained positioning advantage compounded.

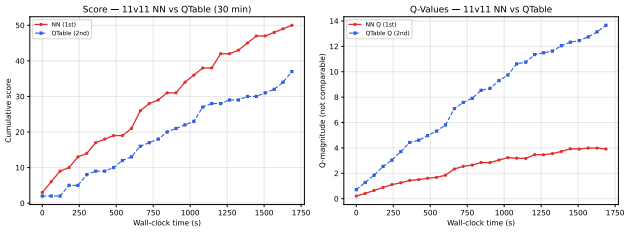

Figure 9: NN vs. QTable at 11v11. NN wins 50–37; narrowest margin across team sizes.

Discussion

Table 1 consolidates all experimental results. The neural network wins every direct comparison, the warm-started policy wins every evaluation, and the NN's advantage in self-play is largest at smaller team sizes where spatial generalization matters most.

The A/B margin peaks at 5v5 (+35) and narrows at 11v11 (+13), consistent with the Q-table's shared-learning advantage growing with player count. The warm/cold margin grows monotonically with team size (+14 → +20 → +30), confirming that prior training compounds as the formation expands. Self-play scores are highest for the NN at 3v3 and 5v5 but converge with the Q-table at 11v11, where the discrete grid saturates and the tactical rules increasingly determine outcomes.

Curriculum Transfer. Weights trained on smaller team sizes bootstrap larger formations (3v3→5v5→11v11). The positional core features (ball and robot coordinates) transfer directly; pairwise distance features are zero-padded for smaller teams and fine-tuned at each new size. This provides a structured initialization for the larger formation, though in the current experiments each team size was trained independently (curriculum transfer remains for future work).

Continuous output. To test whether the 3×3 grid limits performance, we repeated the NN self-play and warm/cold experiments with the continuous-output variant ($k = 2$, direct (x, y) prediction). All other simulation parameters were identical. At 3v3, continuous self-play reached 28–16 (44 goals, Fig. 12) with Q-magnitudes 2.64 and 0.59—far more balanced

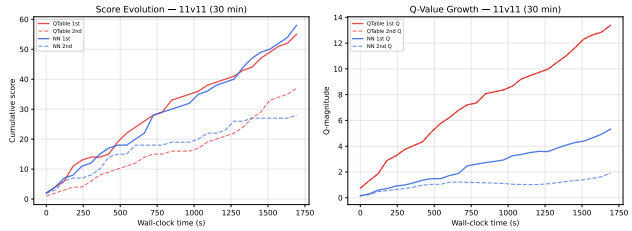

Figure 10: Score and Q-value evolution for 11v11 self-play (30 min). QTable leads with 92 goals; NN reaches 86 with home-team advantage.

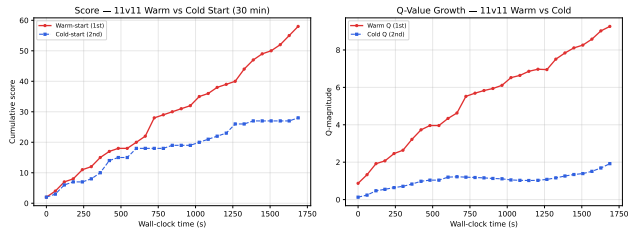

Figure 11: Warm-start vs. cold-start at 11v11. Largest absolute margin (+30); prior training compounds with team size.

than the discrete 225–78 blowout. At 5v5, continuous self-play scored 254–141 (395 goals) and warm/cold 530–297 (827 goals, Fig. 12). At 11v11, continuous self-play reached 282–87 (369 goals) and warm/cold evaluation 261–80 (341 goals), a factor of 3.3× in favor of the trained policy. The continuous variant consistently produces higher total scores than the discrete 9-grid NN (395 vs. 242 at 5v5, 369 vs. 86 at 11v11), while maintaining more symmetric learning between teams.

Continuous vs. discrete head-to-head. To isolate the effect of output resolution, we paired a continuous NN against a discrete 9-grid NN in direct A/B tests. The continuous policy won decisively at all three team sizes: 314–106 at 3v3 (3.0×), 420–240 at 5v5 (1.75×), and 42–18 at 11v11 (2.3×). The discrete NN's Q-magnitude was consistently higher (30.1 vs. 8.6 at 3v3; 62.4 vs. 16.9 at 5v5), yet it lost every match (Fig. 13)—further evidence that Q-values do not cross-compare and that spatial precision, not value magnitude, determines performance.

Taken together, these results confirm that the 3×3 grid is a bottleneck: removing it via continuous output improves both total scoring and competitive balance, and even a 768-parameter network can learn useful continuous positioning given sufficient training.

State space granularity. The 3×3 spatial grid was chosen for simplicity but is almost certainly too coarse for the full complexity of football. At the limit, a 105×68 grid would give the Q-table over 300 million states—the curse of dimensionality. The NN faces no such barrier: replacing discrete outputs with continuous (x, y) removes the quantization bottleneck entirely, matching the CMAPF planner's native discretization.

Table 1: Summary of all experimental results. Scores shown as (winner–loser) for A/B and eval; as (1st–2nd) for self-play. Goal difference Δ reported for asymmetric comparisons.

	3v3 (10 min)		5v5 (20 min)		11v11 (30 min)	
Experiment	Score	Δ	Score	Δ	Score	Δ
NN vs. QTable (A/B)	34–18	+16	83–48	+35	50–37	+13
Warm vs. Cold (eval)	29–15	+14	53–33	+20	58–28	+30
QTable self-play	9–8	—	12–5	—	55–37	—
NN self-play	225–78	—	156–86	—	58–28	—

Feature engineering. The current state representation omits opponent positions (only teammate distances are encoded), ball velocity, and the scoreline. Adding opponent features—even as aggregate statistics like defensive pressure—would allow context-sensitive decisions. The EDMS framework's semantic variables suggest a promising direction.

Training scale. The experiments reported here ran on a single CPU core. The architecture is trivially portable to GPU-accelerated training, where hundreds of parallel game instances could generate millions of goals. The strategy/tactics separation means only the lightweight positioning network needs to be trained; the physics and action resolution can remain CPU-side.

Convergence horizon. Given the sparse reward signal (±5 only on goals), neither policy is likely to converge to expert-level play. However, the consistent relative ordering—NN > Q-table, warm > cold, continuous > discrete—suggests the architectural choices are sound.

From tactics to learning. The actor-critic architecture (`--ppo`) replaces hard-coded tactical rules with NN-driven action selection, with the tactical prior serving as a regularizer that anneals during training. Early results show Q-magnitudes reaching 80+ in continuous self-play, indicating genuine learning. Goals remain relatively rare in early training, and the NN has not yet learned advanced patterns like passing into space or coordinated off-ball movement. The value head's dense training is the most promising path forward: an accurate value function makes advantage estimates meaningful, which makes policy gradients effective. The next step is scaling training time and adding spatial features (grid-based occupancy) so the NN can perceive open space.

Toward realistic play. The policies reported here are early-stage. The actor-critic architecture with dense value training and annealed tactical prior represents a structural improvement over the original "strategy proposes, tactics disposes" design, but training scale remains the bottleneck. Several advances are underway: the 768-parameter network ports trivially to GPU-accelerated environments; the rich2 feature set (37 inputs including velocities) gives the NN spatial momentum awareness; pass-leading and offside-aware targeting

improve attacking patterns; and the simplified 2D shot model with GK reaction time provides cleaner learning signal than KPH-threshold systems.

Zone priors from professional data. We downloaded 435 professional matches from the StatsBomb open-data repository, extracting 1.46 million ball events and 400,000 player freeze-frame positions. From these we computed 7×5 zone priors for ball presence, pass destinations, and player positions, which seed the strategy net's cold-start positioning via the heatmap bias functions. We also derived a pass transition network (which zones do professionals target from each origin?) and shot conversion rates (24% at 5 m declining to 3% at 30 m). The pass network reveals patterns our simulated play currently lacks—notably 40% cutback passes from the final third zones and a systematic forward bias that decays from 73% in defense to 36% in attack. The shot conversion data provides a principled threshold for the killer instinct logic: shoot when conversion exceeds 8% (inside ∼20 m), pass otherwise. These priors load at server startup via `statsbomb_prior.py` and feed into the heatmap bias functions used by the strategy net.

In a direct 11v11 comparison (NN-continuous, identical architecture), the prior-equipped team led 135–54 after 136 s without offside ($Q = 15.62$ vs. 6.03, 2.5×). With offside enabled, the gap widened to 75–15 after 202 s ($Q = 15.17$ vs. 1.90, 8×). Professional priors encode offside-aware defensive lines and timed attacking runs; the no-prior policy, lacking this structure, struggles to adapt when the offside rule restricts naive forward positioning. Notably, the prior advantage grows with training: early on the gap is modest (12–6 after 4 min, $Q = 1.34$ vs. 0.66), but as both policies improve, professional-inspired positioning increasingly pays off.

Pretraining on human data: beyond static zone priors, tracking datasets (StatsBomb, Opta, Metrica) provide per-player (x, y) at 25 Hz for thousands of professional matches; the NN can be warm-started by supervised learning to predict "where would a professional go here?" before RL fine-tuning. *Better physics*: the current ball model omits spin, swerve, and surface-dependent bounce, making passes feel more like billiards than football; calibrating against measured kick data from the physical robots would close the sim-to-real gap. *Tactical depth*: coaching literature—from Wilson's *Inverting the Pyramid* to Cox's *The Mixer*—formalizes the principles we have only begun to mine (pressing triggers, defensive line discipline, attacking width). These can extend the knowledge-mined tactical layer with situationally-aware rules while the RL policy matures.

From Simulation to Physical Robots

The results of Section 5 establish that neural policies outperform tabular methods for spatial positioning. We now turn to the deployment architecture: how these policies transfer to the Unitree G1 humanoid, and how the PPO algorithm connects the soccer strategy layer to the motor-control layer.

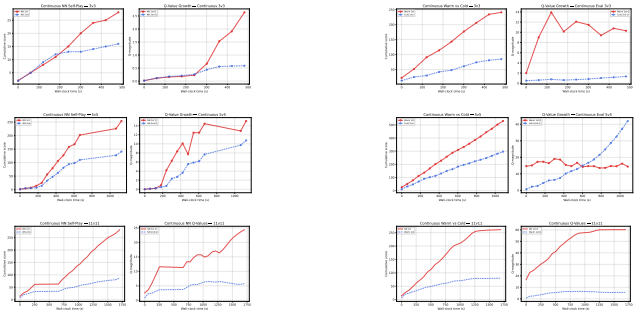

Figure 12: Continuous NN self-play (left) and warm/cold evaluation (right) at 3v3, 5v5, and 11v11. Continuous output removes the 3×3 grid bottleneck, producing higher scores while maintaining competitive balance.

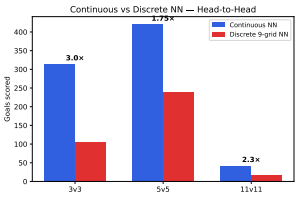

Figure 13: Continuous vs. discrete 9-grid NN. Continuous wins at all three team sizes (3.0×, 1.75×, 2.3×).

Actor-Critic PPO for On-Robot Deployment

The transition from hand-crafted tactics to learned action selection is the bridge from simulation experiments to physical robot deployment. On the Unitree G1 humanoid, every action—shooting, passing, tackling, dribbling—requires precise motor control via the SONIC library. Hard-coding thresholds for when to trigger each skill does not scale across opponents and formations. The actor-critic architecture replaces the fixed tactical decision tree with a lightweight neural policy that the robot can continue to fine-tune onboard.

The network (activated via `--ppo`) has three heads sharing a common feature trunk:

- **Movement head** (2 outputs): continuous (x, y) target, blended 50/50 with the Kohonen strategy net's formation position. The blend gives the NN control over positioning while keeping players tethered to the formation shape.
- **Action head** (4 outputs): logits for shoot, pass, dribble, and move. The final distribution is a softmax over $\text{logit}_{\text{NN}} + \lambda \cdot \text{logit}_{\text{prior}}$, where the tactical prior weight λ anneals from 1.0 to 0.3 over 40 game-increments.
- **Value head** (1 output): scalar $V(s)$ estimating expected future return, trained with TD(0) at every planning cycle. This provides the dense learning signal that sparse goal rewards alone cannot supply.

The tactical prior encodes three coaching principles as cheap hand-crafted logit offsets:

1. **Counter-attack.** In the team's own half with teammates ahead of the ball, forward pass and dribble logits are boosted (+2, +1) while shooting is suppressed. This prioritises quick vertical transitions over slow build-up.
2. **Gegenpressing.** When possession is lost, the three teammates nearest the ball form a pressing squad with boosted approach actions. This provides the structured counter-pressure that prevents the opponent from building play after a turnover.
3. **Overload to switch.** When teammates cluster on one flank and the far side is open, the pass logit is boosted toward the weak side, encouraging rapid ball circulation to exploit defensive compaction.

Early in training ($\lambda = 1.0$), the prior dominates—producing coherent, rule-guided football while the NN's weights are random. As λ anneals, the NN's learned preferences increasingly override the prior. The prior is a teacher, not a constraint: the policy can discover its own pressing triggers, invent new attacking patterns, or learn to ignore the prior's suggestions when they prove suboptimal.

Per-timestep value training. At each planning cycle, every robot computes a TD(0) target $0.001 + \gamma V(s_{t+1})$ and updates only the value-head weights. This yields ∼33,000 value updates per game-minute versus ∼50 for event-driven policy updates, giving the critic a stable baseline for advantage estimation.

Event-driven policy training. The action and movement heads update on significant events: goals (±5), shots on target (+0.5), off-target shots (penalty scaling with distance), completed passes (+0.25 with bonuses for box and wing deliveries), failed passes (−0.25), won tackles (+0.15), fouls (−0.15), and ball-in-box presence (+0.5 per 0.75 s). On goal events, the system replays the preceding trajectory using policy gradient on action logits and value loss on TD targets.

State features. The rich2 feature set (37 dimensions, Table 2) provides the NN with awareness of spatial relationships, runs, pressing intensity, and momentum beyond static positions.

Table 2: The 37-dimensional rich2 input vector. All distances are normalised to $[0, 1]$; velocities are divided by 7 m/s; the ball speed feature uses 30 m/s as the reference. The network outputs a 7-vector: movement target (x, y) in metres, four action logits (shoot, pass, dribble, move), and a scalar value $V(s)$.

#	Feature	#	Feature
1–2	Ball (x, y)	3–4	Robot (x, y)
5	Robot ID (scaled)	6–15	10 nearest teammate distances
16–25	10 nearest opponent distances	26	Distance to opponent goal
27	Ball speed (norm.)	28	Possession flag (0/1)
29–32	Velocity of 2 nearest teammates	33–34	Velocity of nearest opponent
35	Ball direction relative to attack	36–37	Own velocity (x, y)

Algorithm 3 PPO-Clip with tactical prior annealing.

Initialize policy π_θ (3 heads), value V_ϕ, prior weight $\lambda = 1.0$
Set $\gamma = 0.99$, $\varepsilon_{\text{clip}} = 0.2$, $\lambda_{\text{GAE}} = 0.95$, buffer $\mathcal{D} = \emptyset$
for each game step **do**
  Observe state s (37-dim rich2 features)
  Compute tactical prior logits $\ell_{\text{prior}} = \text{tactical\_prior}(s)$
  $\ell_\pi = \pi_\theta(s)$; blend $\ell = \ell_\pi + \lambda \cdot \ell_{\text{prior}}$
  Sample action $a \sim \text{softmax}(\ell)$; execute
  Store $(s, a, r, \ell_\pi, V_\phi(s))$ in rollout buffer $\mathcal{D}$
  Train value head: $V_\phi \leftarrow V_\phi - \alpha_v(V_\phi(s) - (r + \gamma V_\phi(s')))^2$
  if goal scored **then**
    Compute advantages $\hat{A}_t$ via GAE on $\mathcal{D}$ with λ_{GAE}
    Compute returns $R_t = \hat{A}_t + V_\phi(s_t)$
    for $(s_t, a_t, \ell_t, \hat{A}_t, R_t)$ in minibatches from $\mathcal{D}$ **do**
      $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\text{old}}(a_t|s_t)$
      $\mathcal{L}_{\text{clip}} = \mathbb{E}[\min(r_t\hat{A}_t,\ \text{clip}(r_t, 1\pm\varepsilon_{\text{clip}})\hat{A}_t)]$
      $\mathcal{L}_{\text{value}} = \mathbb{E}[(V_\phi(s_t) - R_t)^2]$
      $\mathcal{L} = -\mathcal{L}_{\text{clip}} + 0.5\mathcal{L}_{\text{value}} + 0.01\mathcal{H}(\pi_\theta)$
      $\theta \leftarrow \theta - \alpha\nabla_\theta\mathcal{L}$; $\phi \leftarrow \phi - \alpha_v\nabla_\phi\mathcal{L}_{\text{value}}$
    end for
    Clear buffer $\mathcal{D}$
    $\lambda \leftarrow \max(0.3,\ \lambda - 0.0175)$ ▷ anneal prior over 40 games
  end if
end for

Algorithm 3 (PPO-Clip) differs from standard actor-critic in two respects. First, the clipped surrogate objective prevents destructively large policy updates, stabilising learning with the sparse and noisy goal signal. Second, the tactical prior logits are added to the network's output before the softmax; the annealing schedule starts at $\lambda = 1.0$ (prior and NN equally weighted) and decays to 0.3, at which point the NN contributes roughly twice the prior's influence. The value head trains on every step via TD(0), while the policy head trains only on goal events using stored trajectories, keeping the compute cost low enough for embedded deployment.

Layered PPO: Soccer Strategy on Top of Motor Control The architecture forms a two-level PPO stack (Fig. 14), both layers trained with the same clipped-surrogate objective (Algorithm 3) but operating at different temporal and spatial scales:

The soccer PPO (top layer, this paper) outputs a world-frame movement target (x, y) and a discrete action (shoot/pass/dribble/move) at ∼20 Hz. The SONIC motor PPO (bottom layer) receives this target and executes the corresponding motor skill—a locomotion gait to reach (x, y), a kick trajectory to strike the ball, or a lateral lunge to tackle—at 200 Hz joint-space control. The dashed feedback arrows close the loop: SONIC reports kinematic feasibility ("can I reach that target within the step budget?") back to the soccer layer, allowing the soccer PPO to learn which targets

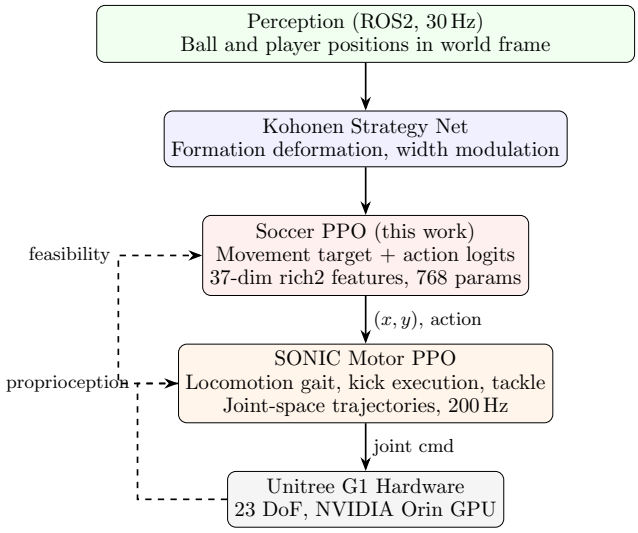

Figure 14: Layered PPO architecture: soccer strategy policy (top) feeds world-frame targets to the SONIC motor-control PPO (bottom), with feasibility feedback flowing upward and proprioception providing motion state.

are physically achievable on the G1 hardware. Proprioception (joint angles, IMU) flows upward, giving the soccer policy awareness of the robot's current motion state.

Both layers share the same PPO-Clip training algorithm with GAE advantage estimation, enabling joint training: the soccer policy can be fine-tuned in simulation with a learned motor model that approximates SONIC's capabilities, then deployed to the physical robot where the motor layer continues to adapt. This layered design isolates domain complexity—the soccer policy never sees joint angles, and the motor policy never sees opponent positions—while the clipped-surrogate objective provides the stable gradient signal needed for both layers to co-learn from sparse, delayed rewards (goals).

Physical Robot Deployment

The primary deployment target is a team of three Unitree G1 humanoid robots competing in the RoboCup AdultSize Humanoid League, with two Unitree Go2 quadruped goalkeepers enabling 4v4 matches. Perception uses servo-controlled cameras with a lightweight neural object detector at 30 Hz via ROS2; the constrained soccer environment (uniform green field, white ball, colored jerseys) allows the detection model to run on CPU.

The state space is designed for direct transfer to physical hardware. The vision pipeline (lightweight NN-based ball and player detection) outputs world-frame positions via ROS2, which are discretized to the same grid used during training. The trained Q-table (JSON) or NN weights (NPZ) are loaded onto each robot's onboard computer. The soccer brain (strategy, tactics, perception) runs entirely on CPU, reserving the onboard GPU for SONIC motor control and self-trained motion-capture locomotion models.

Robot locomotion to policy-selected target positions uses WebRTC for real-time communication between the vision host and the robot controller. Ball kicking is handled by the SONIC motor control library for precise strike execution. Tackling is implemented through SONIC-triggered lateral lunge motions with stability recovery. Trick moves (feints, step-overs) are registered as parameterized skill primitives callable by the policy when dribbling under pressure. CMAPF planning generates collision-free trajectories between waypoints.

Key sim-to-real considerations:

1. **Role consistency**: The state includes a role identifier (0=GK, 1=Def, 2=Fwd) assigned at deployment. Go2 goalkeepers receive role 0; G1 outfield players receive roles 1–2.
2. **Skill calibration**: Simulated skill parameters (passing, shooting, dribbling) will be calibrated against measured robot capabilities from WebRTC-tracked movement and SONIC kick data.
3. **Latency**: The 20 Hz simulation rate accommodates real sensing and actuation delays. Policy inference (Q-table lookup or NN forward pass) runs in under 1 ms on embedded hardware.
4. **Curriculum**: Training begins in 3v3 simulation, transfers to 4v4 (3 G1 + 1 Go2 per team), and ultimately targets full RoboCup matches.

Conclusion

We have presented a complete pipeline for learning robot soccer policies, from physics-based simulation through reinforcement learning to deployment on physical Unitree G1 humanoid robots. The architecture rests on three pillars: a Kohonen-inspired strategy network that deforms formations in response to the ball, a 3-headed actor-critic PPO policy that learns both positioning and action selection, and a physics layer where all outcomes—shots, passes, saves, tackles—are determined by the simulation rather than by probabilistic awards.

On the algorithmic side, neural policies substantially outperform tabular Q-learning for spatial positioning (Section 5), driven by the continuous generalisation of pairwise-distance features. The actor-critic PPO policy (Section 6) extends this to action selection, replacing hand-crafted tactical rules with a learned distribution over shoot, pass, dribble, and move, regularised by an annealed tactical prior. The per-timestep TD(0) value training provides the dense learning signal that sparse goal rewards alone cannot supply, while PPO-Clip with GAE stabilises policy updates.

All policy variants—Q-table (5 KB), neural Q-network, and PPO (768 parameters)—execute forward passes in under a millisecond on embedded CPUs, leaving the onboard GPU (NVIDIA Orin) free for the

SONIC motor-control library and self-trained motion-capture locomotion models. This is essential for the G1 platform: soccer decisions at 20 Hz must not compete with 200 Hz joint-space motor control.

The layered PPO architecture (Fig. 14) connects the soccer strategy layer to the SONIC motor layer through a clean interface: world-frame (x, y) targets and discrete action codes flow downward, feasibility feedback and proprioception flow upward. Both layers share the same PPO-Clip algorithm, enabling joint fine-tuning where the soccer policy learns which targets are physically achievable on the hardware. The policy management system with curriculum transfer (2v2→3v3→5v5→11v11) and browser-driven live swapping supports the experimental workflow needed to iterate on this layered design.

Current limitations point to clear next steps. The PPO policy has not yet been trained at scale; early results show Q-magnitudes reaching 80+ in continuous self-play, but advanced patterns like passing into space and coordinated off-ball movement remain to emerge. The value head's dense training is the most promising path forward, and scaling to GPU-accelerated environments with hundreds of parallel game instances is the immediate priority. Professional tracking data (StatsBomb, Opta) offers a path to warm-start the policy via behavioural cloning before RL fine-tuning. On the hardware side, calibrating the simulated skill parameters (passing, shooting, tackling) against measured robot capabilities will close the sim-to-real gap for the SONIC motor layer.

The separation of concerns at the heart of this architecture—strategy net proposes where to stand, PPO learns what to do and refines where to go, physics determines outcomes—keeps each layer tractable while producing competent football from sub-kilobyte policies. This design generalises beyond soccer: any multi-agent spatial task can substitute domain-specific action models and motor controllers while preserving the Kohonen positioning and PPO learning layers.

References

[1] RoboCup Humanoid League 2024, *AdultSize Rules and Regulations*, online.

[2] RoboCup Humanoid League 2025, *Rule Draft Update*, online.

[3] M. Abreu, L. P. Reis, N. Lau, "Designing a Skilled Soccer Team for RoboCup: Exploring Skill-Set-Primitives through Reinforcement Learning," *Neural Computing and Applications*, 2025.

[4] A. Raichuk et al., "Google Research Football: A Novel Reinforcement Learning Environment," *arXiv:1907.11180*, 2019.

[5] "PES 2021 dt18 Constants Deep Dive," *implyingrigged.info/wiki/User:Aoba/PES2021_dt18*.

[6] "PES 2021 dt18 Files Breakdown (MarcoZ)," *implyingrigged.info/wiki/User:MarcoZ/dt18_Files_Breakdown*.

[7] StatsBomb, "Contextual Expected Threat (xT) Using Spatial Event Data," *StatsBomb Research*, 2022.

[8] TrueSim 1.5, "A True Simulation Project (Stats & Gameplay Fixed)," *sortitoutsi.net*, 2023.

[9] B-Human, "12-Time World Champion Wins German Cup with Booster K1 Humanoids," *Interesting Engineering*, 2024.

[10] V. Mnih et al., "Human-Level Control through Deep Reinforcement Learning," *Nature*, 518(7540):529–533, 2015.

[11] T. Mikkola, R. de Jong, A. Kanervisto, and J. M. S. Garcia, "Goalkeeper Spatial Positioning in EA Sports FC 25 via Deep Reinforcement Learning," *arXiv:2510.23216*, 2025.

[12] P. Rahimian, J. Van Haaren, and L. Toka, "A Deep Reinforcement Learning Approach to Optimal Ball Destination in Soccer from Tracking Data," *Int. J. Sports Science & Coaching*, 19(1):230–244, 2024.

[13] P. Rahimian, J. Van Haaren, and L. Toka, "Beyond Spatial Coordinates: Spatio-Semantic State Representations for Improved Off-Ball Football Positioning with Deep Reinforcement Learning," *arXiv:2510.00480*, 2024.

[14] MFF UK Robot Lab, "Roboti z Matfyzu přivezli stříbro z Vídně" (Robots from Math-Physics Faculty Bring Silver from Vienna), *Charles University*, 2026.